Worked examples

These examples connect the model to observable behavior: following one job, choosing a useful capacity upgrade, executing a coupled QA-to-DQC workflow, and reading the archived paper results. Examples 1-3 run locally with the input JSON files in **examples/**. Example 4 uses the public dataset.

1. Follow one job from preparation to completion

The input uses two CPU devices, one DQC controller, and one 32-qubit DQC device. Its assumed durations are deliberately simple. The job arrives at time zero and executes five dependent stages.

Stage	Resource	Service (s)	Start / finish (s)
Preparation	CPU	1	0 / 1
Controller staging	DQC controller	4	1 / 5
Quantum execution	DQC device	2	5 / 7
Controller readout	DQC controller	1	7 / 8
Postprocessing	CPU	1	8 / 9

```
python -m hybridqc.cli.hcu_sim \
  --config examples/baseline-config.json \
  --jobs examples/one-job.json \
  --execution microstep \
  --output runs/tutorial-one.json
```

Expected: jobs_submitted = jobs_completed = 1, total_time_s = 9.0, and every stage_wait_time_s entry is zero. The five services sum to 9.0 s because this single job has no overlapping stages or resource competitors.

Open reservation_trace to check each start and finish against the table. The DQC stage uses all 32 qubits, and controller staging and readout use the same controller. CPU work before and after quantum execution remains explicit.

What to change next

Increase one duration_s and rerun to a new file. For this single sequential job, increasing DQC duration from 2 to 3 s increases completion from 9 to 10 s. Adding another otherwise idle device does not shorten an individual service.

All tutorial service times are assumed. Transfers and physical hardware acquisition are outside these teaching cases. They are not additional paper measurements.

2. Identify the useful resource upgrade

Run the same eight jobs, all arriving at zero, against four configurations. Each job uses the five stages from Example 1. The cases change only DQC and controller counts.

```
for variant in baseline two-qpus two-controllers balanced
do
  python -m hybridqc.cli.hcu_sim \
    --config "examples/$variant-config.json" \
    --jobs examples/eight-jobs.json \
    --execution microstep \
    --output "runs/tutorial-$variant.json"
done
```

Configuration	QPU / controller count	Completed jobs	Final finish (s)
Baseline	1 / 1	8 / 8	54.0
Two QPUs	2 / 1	8 / 8	54.0
Two controllers	1 / 2	8 / 8	35.0
Both doubled	2 / 2	8 / 8	35.0

Doubling controller capacity reduces elapsed time by 19 s, a 54/35 = 1.54x speedup. Adding only a QPU leaves this case unchanged. The configured schedule needs more controller capacity before extra quantum capacity is useful.

Check the mechanism in the output

Summed stage waiting (s)	Baseline	Two controllers
Controller staging	116	52
Controller readout	59	45
DQC execution	136	72
All stages	344	192

The stage-service totals stay fixed at 72 s across the eight jobs. Controller staging contributes 32 s and readout 8 s of that work. The runtime improvement comes from the schedule and waiting. Summed waits overlap across jobs, so 344 s is not the elapsed runtime.

The result is specific to these tasks, durations, arrivals, and FIFO policy. A useful next experiment changes the DQC service duration while leaving the controller work fixed, then repeats the same four configurations to locate a different bottleneck.

Checked values are included in examples/expected/. The teaching result is independent of the archived paper comparisons.

3. Execute a QA-to-DQC workflow

A coupled workflow uses quantum annealing to produce an intermediate result, processes that result on the CPU, and then executes a DQC stage. The input has separate QA and DQC controllers, so each path has its own finite service resource.

Task order	Resource	Assumed service (s)
Preparation	CPU	1.0
QA programming	QA controller	0.5
QA execution	128-qubit QA device	1.0
QA readout	QA controller	0.5
Intermediate processing	CPU	1.0
DQC staging	DQC controller	4.0
DQC execution	32-qubit DQC device	2.0
DQC readout	DQC controller	1.0
Postprocessing	CPU	1.0

```
python -m hybridqc.cli.hcu_sim \
  --config examples/hybrid-config.json \
  --jobs examples/hybrid-jobs.json \
  --execution microstep \
  --output runs/tutorial-hybrid.json
```

Expected: both submitted jobs complete. Job 1 finishes at 14.0 s and job 2 at 16.0 s. The mean job latency is 15.0 s and total_time_s is 16.0 s. Each of the nine stage types completes twice.

Each job has 12 s of service. The extra time comes from waiting for shared resources. The CPU intermediate stage follows QA readout, and DQC staging follows that intermediate stage. Inspect task_idx and job_id in the trace to follow these dependencies while the two jobs overlap.

Extend the example deliberately

To study communication, supply the directed host/controller/device routes and corresponding transfer tasks with nonzero payloads. To study a different application, change its stage sequence and service inputs while keeping the machine fixed. Keep measured device service and assumed architectural costs identifiable in the experiment input.

4. Read a published architecture comparison

Download **hybridqc-supplementary-data-v1.0.0.zip** from Zenodo and extract it. This example uses the CSV data and Python's standard library, so it requires no simulator installation.

Open a terminal in the extracted directory that contains DATA_CATALOG.csv. The archive groups datasets by topic. For a portfolio topology comparison, read topology_and_scheduling/topology_comparisons.csv.

```
python - <<'PY'
import csv
from pathlib import Path
p = Path("topology_and_scheduling/topology_comparisons.csv")
with p.open(newline="") as f:
    rows = [r for r in csv.DictReader(f)
            if r["workload"] == "portfolio"]
times = {r["topology"]: float(r["runtime_s"]) for r in rows}
base = times["baseline"]
for name, seconds in times.items():
    print(f"{name:18s} {seconds:8.3f} s  {base/seconds:.2f}x")
PY
```

Topology	Runtime (s)	Speedup over baseline
Baseline	66.324	1.00x
DQC-only	37.470	1.77x
QA-only	66.324	1.00x
Faster-link-only	66.324	1.00x
Balanced 10x	26.857	2.47x

For this workload, DQC capacity helps, while QA-only and faster-link-only upgrades do not change completion. Balanced scaling produces a further gain. This is a different bottleneck from the small controller-heavy tutorial in Example 2.

Choose the next comparison

Use policy_comparisons.csv in the same directory for scheduling within a workload. Use cloud_execution/execution_mode_comparison.csv for deployment overhead. Use workload_scaling/ for named growth axes and completion counts. DATA_DICTIONARY.md defines field units and approximation markers.

Keep comparisons within the same study and workload. A median over configurations, a mean over jobs, and the final finish time of one run answer different questions. Retain the dataset filename and selected configuration labels alongside any derived speedup.